# Data Governance Copilot

Interview Question Bank

Topic 03 — LLM Integration & Fallback

12 questions · 4 sections · LiteLLM · Structured Output · Intent Classification · _MockLLM · Token Budget

ndational Core concepts | Intermediate Design decisions | Advanced Deep internals | Gotcha Real bugs / traps

---

## LiteLLM & the Fallback Chain

**Q01** Foundational

### What is LiteLLM and why did you choose it over calling the Groq or Anthropic SDK directly?

*Hint: Think about provider independence and fallback orchestration.*

**Key points to cover:**

- LiteLLM is a unified LLM proxy library — one interface (OpenAI-compatible) for 100+ providers
- Calling Groq SDK directly hard-codes the provider; switching to Bedrock or Anthropic needs code changes
- LiteLLM abstracts provider differences: same .invoke() call works for Groq, OpenAI, Anthropic, Gemini
- Built-in fallback orchestration: if primary model fails, LiteLLM tries the next in fallback_models list automatically
- Also provides token usage tracking, cost calculation, and request logging — useful for llm_guard budget enforcement
- In your project: get_llm(config) returns a ChatGroq wrapper but with LiteLLM's fallback chain configured underneath
- AWS Bedrock migration (P0 item) is a one-line config change — model string swap, no application code changes

Q02 Intermediate

### Walk through the full fallback chain in your project. What triggers a fallback and what order does it try providers?

*Hint: Trace from primary failure to final fallback.*

**Key points to cover:**

- Primary: Groq (llama-3.3-70b-versatile) via ChatGroq — fastest, cheapest, no egress from your main workflow
- Fallback order: gpt-4o-mini → anthropic/claude-haiku-4-5 → gemini/gemini-1.5-flash
- Fallback triggers: HTTP 429 (rate limit), 503 (service unavailable), timeout, or any provider-side exception
- LiteLLM catches the exception, logs it, and immediately retries with the next model in fallback_models list
- If ALL providers fail: _MockLLM stub is returned — always returns a string, never raises (last-resort safety net)
- No GROQ_API_KEY set at all: get_llm() returns _MockLLM directly without attempting any real provider
- Structured output path: get_structured_llm() wraps the fallback chain with .with_structured_output(schema) — fallback still applies

```
# LLMConfig fallback_models list fallback_models = [ 'gpt-4o-mini', 'anthropic/claude-haiku-4-5', 'gemini/gemini-1.5-flash', ] # Primary llm = ChatGroq(model='llama-3.3-70b-versatile') # With fallbacks wired llm_with_fallbacks = llm.with_fallbacks( [LiteLLM(model=m) for m in fallback_models] )
```

Q03 Advanced

### Your P0 production item is swapping Groq for AWS Bedrock (Claude 3.5 Sonnet). What exactly changes in the codebase and why is it truly 'one line'?

*Hint: This tests whether you understand LiteLLM's abstraction depth.*

**Key points to cover:**

- LiteLLM model string for Bedrock: 'bedrock/anthropic.claude-3-5-sonnet-20241022-v2:0'
- Change LLM_PROVIDER=groq → LLM_PROVIDER=bedrock and LLM_MODEL=anthropic.claude-3-5-sonnet in env vars
- llm_factory.py reads config and constructs the appropriate client — no agent or node code changes needed
- Bedrock auth uses IAM role (ECS task role) — no API key needed; LiteLLM picks up boto3 credentials automatically
- Why it matters: data never leaves AWS VPC — no Groq egress, no external SLA dependency
- Potential gotcha: Bedrock response format differs slightly — .with_structured_output() must still work; test with IntentClassification schema
- Also update fallback chain: Bedrock primary → gpt-4o-mini (via PrivateLink or keep as-is) → gemini as last resort

Q04 Gotcha

### If your primary Groq model is rate-limited and falls back to claude-haiku, but claude-haiku also fails, what does the user see? Walk through the complete error path.

*Hint: Think about what _MockLLM returns and how the synthesizer handles it.*

**Key points to cover:**

- LiteLLM tries Groq → gpt-4o-mini → claude-haiku → gemini in sequence
- If all four fail: get_llm() returns _MockLLM — a stub that always returns a hardcoded string like 'Mock LLM response'
- _MockLLM never raises — it satisfies the BaseChatModel interface so the graph continues executing
- synthesizer_node receives _MockLLM output as final_summary — it's a meaningless stub string
- User sees a degraded but non-crashing response: 'Mock LLM response' instead of a real synthesis
- Better production behaviour: detect _MockLLM is active and return a user-friendly error message ('Service temporarily unavailable')
- llm_guard token budget check runs before LLM call — if budget exceeded, the call is blocked before even trying providers

## Structured Output & Pydantic V2

**Q05** Foundational

## What is structured output in LLM terms and how does .with_structured_output() work with Pydantic V2?

*Hint: Contrast with asking the LLM to 'respond in JSON'.*

**Key points to cover:**

- Structured output: the LLM is constrained to return a response matching a specific schema — not free-form text
- Provider-side mechanism: OpenAI/Groq use JSON mode or function calling under the hood to enforce the schema
- .with_structured_output(IntentClassification) wraps the LLM chain — output is automatically parsed into a Pydantic model
- Pydantic V2 provides fast validation, clear error messages, and field-level type coercion
- Contrast with 'respond in JSON': LLM may hallucinate wrong keys, miss required fields, or add prose before the JSON
- With structured output + Pydantic: parsing failure raises ValidationError immediately — triggerable fallback
- In your project: get_structured_llm(config, IntentClassification) → used in intent.py for every query classification

```
# IntentClassification schema class IntentClassification(BaseModel): intent: QueryIntent # str Enum — 10 values data_products: List[str] # ['retention', 'bookings'] confidence: float # 0.0 - 1.0 reasoning: str # one-sentence rationale # Usage chain = get_structured_llm(config, IntentClassification) result: IntentClassification = chain.invoke(prompt)
```

**Q06** Intermediate

## Why is QueryIntent defined as a str Enum with 10 values rather than a plain string field? What does this buy you?

*Hint: Think about what happens at the LLM prompt level and the routing level.*

**Key points to cover:**

- str Enum restricts the LLM to exactly 10 valid intent values — it cannot hallucinate 'data_lineage' or 'compliance_check'
- LiteLLM/Groq with structured output uses the Enum values in the JSON schema sent to the model — provider enforces valid values
- At routing layer: INTENT_AGENT_MAP keys are exactly the Enum values — no KeyError possible from unexpected intent string
- Pydantic V2 validates the Enum on parse: if LLM returns an invalid string, ValidationError is raised immediately
- ValidationError → triggers _keyword_fallback() — graceful degradation without crashing the graph
- str Enum also provides human-readable values (not int codes) — logs and traces are self-documenting
- 10 intents cover all routing paths: full_diagnostic, data_quality, governance, incident_review, knowledge_lookup, metric_analysis, write_ticket, write_metadata, write_rule, unknown

**Q07** Advanced

## The confidence float field in IntentClassification ranges 0.0-1.0. How do you use it downstream and what would you do if confidence is very low?

*Hint: Think about routing decisions and user experience.*

**Key points to cover:**

- confidence is written into AgentState and included in the final_summary for transparency
- Low confidence (e.g. < 0.4) on 'unknown' intent routes to ['information', 'knowledge'] — broadest safe fallback
- You could gate HITL on confidence: if confidence < 0.5 and write intent → require human approval before executing
- Confidence in the API response (/query JSON) lets the client UI show a 'low confidence' warning to the user
- Improvement: if confidence < 0.3, return a clarification request to the user rather than routing to agents
- reasoning field provides the one-sentence rationale — useful for debugging misclassifications in LangSmith traces
- Monitoring: track confidence distribution in CloudWatch — a drop in average confidence signals prompt drift or data shift

## Intent Classification & Keyword Fallback

**Q08** Intermediate

### Describe the full intent classification flow. What happens when the LLM is unavailable?

*Hint: There are two completely different classification paths — explain both.*

**Key points to cover:**

- Primary path: get_structured_llm(config, IntentClassification).invoke(prompt) → Pydantic-validated IntentClassification
- Prompt includes: query text, available data products, and examples of each intent — few-shot style
- Failure triggers for fallback: no GROQ_API_KEY, all providers rate-limited, ValidationError from bad LLM output
- _keyword_fallback(query) is the secondary path — pure string matching, no LLM call
- KEYWORD_MAP: dict mapping intent name → list of trigger keywords (e.g. 'data_quality' → ['quality', 'dq', 'missing', 'null'])
- Fallback returns IntentClassification with confidence=0.5, reasoning='keyword match' — signals degraded mode
- Final safety net: if no keyword matches, returns intent='unknown', confidence=0.0 → routes to broadest agent set

```
# KEYWORD_MAP structure (illustrative) KEYWORD_MAP = { 'data_quality':
['quality','dq','missing','null','duplicate'], 'governance':
['policy','rule','compliance','regulation'], 'incident_review':
['incident','outage','sla','ticket','jira'], 'write_rule': ['create rule','add rule','define rule'],
# ... 6 more intents }
```

**Q09** Gotcha

### In your test suite, _keyword_fallback('create rule for retention') was used to fix Bug #42. What was the original bug and why did the fix work?

*Hint: This is about keyword specificity and ordering in the KEYWORD_MAP.*

**Key points to cover:**

- Original test: _keyword_fallback('create rule for data quality') — expected 'write_rule' intent
- Bug: 'data quality' keywords matched 'data_quality' intent before 'write_rule' keywords were checked
- Root cause: KEYWORD_MAP iteration order — 'data_quality' entry came before 'write_rule'; 'data quality' substring matched first
- Fix: change test query to 'create rule for retention' — 'retention' is not a data_quality keyword, so only 'write_rule' matches
- Deeper fix: keyword matching should check more specific (longer) patterns before generic ones — 'create rule' is more specific than 'quality'
- Lesson: keyword fallback is inherently fragile — it is a last-resort, not a primary classification strategy
- Production implication: monitor intent distribution — if fallback rate > 5%, the LLM prompt needs tuning

## _MockLLM & llm_guard Token Budget

**Q10** Intermediate

## What is _MockLLM and what interface does it implement? Why is implementing the full BaseChatModel contract important?

*Hint: Think about duck typing and test isolation.*

**Key points to cover:**

- _MockLLM is a stub class that implements LangChain's BaseChatModel interface — same .invoke(), .stream() signatures
- Always returns a fixed AIMessage with content='Mock LLM response' — never raises, never makes network calls
- Activated when: GROQ_API_KEY is missing OR all real providers have failed
- Full BaseChatModel contract means: any code that accepts a BaseChatModel works with _MockLLM without modification
- In tests: patch get_llm() to return _MockLLM → agents and synthesizer run without any API keys or network
- Without proper interface: code doing llm.invoke() would work but llm.with_structured_output() would raise AttributeError
- Also used to satisfy get_structured_llm() — _MockLLM.with_structured_output() returns a chain that outputs a fake IntentClassification

```
class _MockLLM(BaseChatModel): def _generate(self, messages, stop=None, **kw): return ChatResult(generations=[ ChatGeneration(message=AIMessage(content='Mock LLM response')) ]) def _llm_type(self) -> str: return 'mock' # Must also implement: _stream, bind_tools, with_structured_output
```

**Q11** Advanced

## Explain the llm_guard daily token budget. How does it work and what happens when the budget is exceeded?

*Hint: Think about the enforcement point and persistence of the counter.*

**Key points to cover:**

- llm_guard.py implements a hard daily token budget — prevents runaway LLM costs in production
- get_daily_usage() reads current token count for today's date key from Redis (or in-memory fallback)
- Before every LLM call: check if current_usage + estimated_tokens > DAILY_TOKEN_BUDGET
- If budget exceeded: raises TokenBudgetExceeded exception — the LLM call is blocked entirely, no fallback attempted
- Token counter is incremented after each successful LLM call using actual token count from response metadata
- Redis key pattern: 'token_budget:YYYY-MM-DD' with TTL=86400s — auto-resets at midnight UTC
- P0 production issue: token counters must be in Redis (shared across ECS tasks) — in-memory counter is per-process and will undercount
- get_daily_usage() is exposed via GET /agents/status — ops team can monitor budget consumption in real time

```
# Conceptual llm_guard flow def guarded_llm_call(llm, messages, config): usage = get_daily_usage() # Redis GET token_budget:today estimate = estimate_tokens(messages) if usage + estimate > config.daily_token_budget: raise TokenBudgetExceeded( f'Daily budget {config.daily_token_budget} exceeded' ) response = llm.invoke(messages) increment_usage(response.usage.total_tokens) # Redis INCRBY return response
```

Q12 Design

## A business user complains they got a 'budget exceeded' error at 2pm. How would you diagnose and prevent this in production?

*Hint: This is an operational / architecture question about token budget management.*

**Key points to cover:**

- Diagnosis: check GET /agents/status → daily_token_usage field shows current spend; CloudWatch metric shows usage curve
- Find the spike: LangSmith traces (or X-Ray in prod) show which queries consumed the most tokens — likely full_diagnostic intent routing to 4 agents + synthesizer
- Per-intent budgets: assign lower token limits to intents that don't need expensive multi-agent runs
- Per-user budgets: track token usage by user_id not just globally — prevent one heavy user from blocking everyone
- Soft vs hard limit: at 80% of budget send a warning (Slack alert); at 100% enforce the block
- Dynamic budgets: increase limit for business hours, reduce overnight — most governance queries happen 9-5
- Caching impact: @cached_node means cache hits consume zero tokens — monitor cache hit rate alongside token usage
- Token budget is a business control, not just a cost control — document it in runbook so ops can emergency-increase via env var